

Big Data – TP1 : Algorithmique et programmation distribuée en Spark

Bastien CETRE – Ahmed EL AYEB 3A SDI 25 septembre 2026

Environnement. Nous avons travaillé sur le cluster Spark/HDFS du DCE (NameNode et Spark Master sur `sar17`, compte `sdm_9` (ou `14`)), depuis un nœud `kyle` réservé avec `srunk`, après avoir chargé les modules `openjdk/17`, `hadoop/3.4.0` et `spark/4.2.0`. Tous les programmes sont lancés avec `spark-submit -master spark://sar17:7077 <script> <fichier>`, et les temps sont relevés sur la ligne `Job 0 finished ... took X ms` des logs.

1 Calcul de moyennes

1.1 Version lente

```
def avg_temperature_slow(theText_file):
    temperatures = theText_file \
        .map(lambda line: line.split(",")) \
        .map(lambda term: (term[0], [float(term[6])])) \
        .reduceByKey(lambda x, y: x+y) \
        .mapValues(lambda lv: sum(lv)/len(lv))
    return temperatures
```

Chaque ligne donne un couple (`année`, `[température]`). Le `reduceByKey` ne fait que concaténer des listes : au final, toutes les températures d’une année se retrouvent dans une seule liste sur un seul executor, comme avec un `groupByKey`. On calcule ensuite la moyenne sur cette liste.

```
$ spark-submit --master spark://sar17:7077 avg_temperatures_slow.py temperatures_86400.csv 2>&1 | grep "Job 0 finished" | grep -o "took
.*"
took 2952.352442 ms
$ hdfs dfs -cat hdfs://sar17:9000/sdm/sdm_9/temperatures_86400.out/* | sort | tail -6
('2013', 8.159945205479453) ('2014', 7.416164383561644) ('2015', 7.7883287671232875)
('2016', 8.337049180327869) ('2017', 7.345287671232877) ('2018', 8.235123287671234)
```

On retrouve bien les valeurs de l’énoncé. Pour les autres fichiers, on relance la même commande (après `hdfs dfs -rm -r` du dossier de sortie) : 7181 ms pour 2880 et 79676 ms pour 86. Comme demandé par l’enseignant, nous n’avons pas lancé la version lente sur le fichier de 3 Go.

Les fichiers 2880 et 86 font presque la même taille, mais le second est 11 fois plus lent. La différence vient du nombre de clés : 2880 couvre 39 années, alors que 86 ne contient que 1980. Toutes ses valeurs (environ 367 000) ont donc la même clé et finissent dans une seule tâche de réduction, qui recopie des listes de plus en plus longues à chaque `x+y`. Le calcul n’est plus du tout parallèle et tout le trafic réseau converge vers un seul executor. Ce qui compte ici, c’est le nombre de valeurs par clé, pas la taille du fichier.

1.2 Version rapide

```
def avg_temperature_fast(theText_file):
    temperatures = theText_file \
        .map(lambda line: line.split(",")) \
        .map(lambda term: (term[0], (float(term[6]), 1))) \
        .reduceByKey(lambda x, y: (x[0]+y[0], x[1]+y[1])) \
        .mapValues(lambda s: s[0]/s[1])
    return temperatures
```

Cette fois chaque mesure devient (`année`, `(t, 1)`) et on additionne séparément les sommes et les compteurs. L’addition étant associative et commutative, Spark peut faire la réduction localement dans chaque partition avant le shuffle. Chaque executor n’envoie alors qu’un couple (`somme`, `nombre`) par année. La moyenne est calculée à la fin.

```
$ for f in 86400 2880 86 10; do hdfs dfs -rm -r -f hdfs://sar17:9000/sdm/sdm_9/temperatures_${f}.out; echo -n "temperatures_${f} : "; spark
-submit --master spark://sar17:7077 avg_temperatures_fast.py temperatures_${f}.csv 2>&1 | grep "Job 0 finished" | grep -o "took.*";
done
temperatures_86400 : took 2987.822549 ms temperatures_86 : took 5072.749446 ms
temperatures_2880 : took 3170.88351 ms temperatures_10 : took 22340.409674 ms
```

Les moyennes obtenues sont les mêmes qu’avec la version lente.

Fichier	86400	2880	86	10
Taille (Mo)	0,36	10,5	9,0	3000
Temps version lente, GBK (s)	2,95	7,18	79,68	–
Temps version rapide, RBK (s)	2,99	3,17	5,07	22,34

TABLE 1 – Temps d’exécution des deux versions du calcul de moyenne.

Sur le petit fichier les deux versions se valent (environ 3 s) : à cette taille on ne mesure presque que le coût de lancement d’un job Spark. Sur 2880 on gagne un facteur 2,3, et sur 86 un facteur 16. Même le fichier de 3 Go passe en 22 s.

Le gain vient du shuffle et de la réduction. Le volume envoyé sur le réseau ne dépend plus du nombre de mesures mais seulement du nombre d’années par partition, et la réduction est faite par tous les executors en parallèle au lieu d’une seule tâche. En contrepartie, on crée un tuple `(t, 1)` par ligne et on ajoute une étape de combinaison côté map,

mais ce surcoût est négligeable. Sur le gros fichier, le temps restant correspond surtout à la lecture dans HDFS et au découpage des lignes.

2 Amis communs dans un réseau social

2.1 Conception

Une ligne X , u , v , w veut dire que X est ami avec u , v et w . X est donc un ami commun de tous les couples qu'on peut former avec ses amis. Pour chaque ligne, la fonction f émet les paires $((u, v), X)$ avec $u < v$, ce qui évite d'avoir à la fois (A, B) et (B, A) . Il reste à regrouper les X par couple.

```
def f(t):
    common_friend = t[0]
    l = []
    for u in t[1:]:
        for v in t[1:]:
            if u < v:
                l.append((u, v), common_friend))
    return l

def common_friends_gbk(theTextFile): # version groupByKey
    cf = theTextFile.map(lambda line: line.split(",")) \
        .flatMap(lambda t: f(t)) \
        .groupByKey() \
        .mapValues(lambda t: list(t))
    return cf

def common_friends_rbk(theTextFile): # version reduceByKey
    cf = theTextFile.map(lambda line: line.split(",")) \
        .flatMap(lambda t: f(t)) \
        .mapValues(lambda x: [x]) \
        .reduceByKey(lambda x, y: x + y)
    return cf
```

On utilise `flatMap` parce que f renvoie une liste de paires. Une personne qui a d amis génère $d(d-1)/2$ paires. Le mode (gbk ou rbk) est passé en deuxième argument du script.

```
$ spark-submit --master spark://sar17:7077 common_friends.py sn_tiny.csv gbk 2>&1 | grep "Job 0 finished" | grep -o "took.*"
took 2913.294335 ms
$ hdfs dfs -cat hdfs://sar17:9000/sdim/sdim_9/sn_tiny_gbk.out/* | sort
(('A', 'B'), ['D']) (('A', 'F'), ['D']) (('B', 'E'), ['D']) (('C', 'E'), ['D']) (('D', 'F'), ['E'])
(('A', 'C'), ['D']) (('B', 'C'), ['D', 'A']) (('B', 'F'), ['D']) (('C', 'F'), ['D']) (('E', 'F'), ['D'])
(('A', 'D'), ['C', 'B']) (('B', 'D'), ['A']) (('C', 'D'), ['A']) (('D', 'E'), ['F'])
(('A', 'E'), ['D'])
```

Nous avons vérifié ce résultat à la main (15 couples). La version `rbk` donne les mêmes couples (2952 ms), seul l'ordre dans certaines listes change.

2.2 Mesures

```
$ for f in sn_10k_100k sn_100k_100k sn_1k_100k sn_1m_1m; do for m in gbk rbk; do hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/${f}_${m}
.out; echo -n "$f $m : "; spark-submit --master spark://sar17:7077 common_friends.py $f.csv $m 2>&1 | grep "Job 0 finished" | grep -
o "took.*"; done; done
sn_10k_100k gbk : took 12147.234243 ms sn_1k_100k gbk : took 21104.056698 ms
sn_10k_100k rbk : took 11088.979482 ms sn_1k_100k rbk : took 59075.860425 ms
sn_100k_100k gbk : took 3535.627581 ms sn_1m_1m gbk : took 12286.399615 ms
sn_100k_100k rbk : took 3446.59673 ms sn_1m_1m rbk : took 11687.089959 ms
```

Les deux versions sont proches, sauf sur `sn_1k_100k` où `reduceByKey` est presque 3 fois plus lent. Ce graphe a peu de personnes mais beaucoup de liens : chaque couple a beaucoup d'amis communs, donc les listes sont longues. On retombe sur le problème de la question 1.1 : le $x + y$ recopie les listes à chaque étape, et la réduction locale ne diminue pas le volume envoyé puisque la taille de la valeur n'est pas bornée. Le `groupByKey` construit chaque liste une seule fois. `reduceByKey` n'est donc intéressant que si la valeur réduite garde une taille fixe (somme, compteur, min, max).

2.3 et 2.5 Degrés et nombre réel de paires intermédiaires

Un seul programme calcule tout en une passe sans sortir du RDD. Chaque personne de degré d devient le tuple $(d, d, d, 1, d(d-1)/2)$, puis un `reduce` donne le min, le max, la somme des degrés, le nombre de nœuds et le nombre exact de paires produites par f .

```
degrees = lines.map(lambda line: line.split(",")).map(lambda t: len(t) - 1)
mn, mx, s, n, p = degrees.map(lambda d: (d, d, d, 1, d*(d-1)/2)) \
    .reduce(lambda a, b: (min(a[0], b[0]), max(a[1], b[1]), a[2]+b[2], a[3]+b[3], a[4]+b[4]))
avg = s / n
```

```
$ for f in sn_tiny sn_10k_100k sn_100k_100k sn_1k_100k sn_1m_1m; do spark-submit --master spark://sar17:7077 sn_stats.py $f.csv 2>/dev/
null | grep STATS; done
STATS sn_tiny.csv nodes= 6 min= 2 max= 5 avg= 2.67 est_pairs= 13 meas_pairs= 17
STATS sn_10k_100k.csv nodes= 10000 min= 5 max= 39 avg= 20.0 est_pairs= 1900000 meas_pairs= 1999916
```

STATS sn_100k_100k.csv nodes= 100000 min= 0 max= 12 avg= 2.0 est_pairs= 100000 meas_pairs= 199594
 STATS sn_1k_100k.csv nodes= 1000 min= 153 max= 248 avg= 200.0 est_pairs= 19900000 meas_pairs= 19996548
 STATS sn_1m_1m.csv nodes= 1000000 min= 0 max= 12 avg= 2.0 est_pairs= 1000000 meas_pairs= 2000391

	sn_tiny	sn_10k_100k	sn_100k_100k	sn_1k_100k	sn_1m_1m
Taille (Mo)	$4 \cdot 10^{-5}$	0,98	1,7	0,75	19,7
groupByKey (s)	2,91	12,15	3,54	21,10	12,29
reduceByKey (s)	2,95	11,09	3,45	59,08	11,69
Min / Max nb d'amis	2 / 5	5 / 39	0 / 12	153 / 248	0 / 12
Nb moyen d'amis	2,67	20	2	200	2
Paires intermédiaires estimées	13	1 900 000	100 000	19 900 000	1 000 000
Paires intermédiaires mesurées	17	1 999 916	199 594	19 996 548	2 000 391

TABLE 2 – Récapitulatif de l'exercice 2 (onglet Exo2).

2.4 Estimation du nombre de paires intermédiaires

Si tous les nœuds avaient le degré moyen $\bar{d}$, on aurait $P_{\text{est}} = N\bar{d}(\bar{d} - 1)/2$ paires. L'estimation tient à moins de 5 % près pour **sn_10k_100k** et **sn_1k_100k**, où les degrés sont assez homogènes. Par contre elle est deux fois trop basse pour **sn_100k_100k** et **sn_1m_1m**, où les degrés vont de 0 à 12 pour une moyenne de 2. Comme $d(d-1)/2$ croît comme d^2 , les quelques nœuds à fort degré produisent beaucoup plus de paires que la moyenne ne le laisse penser.

2.6 Analyse des performances

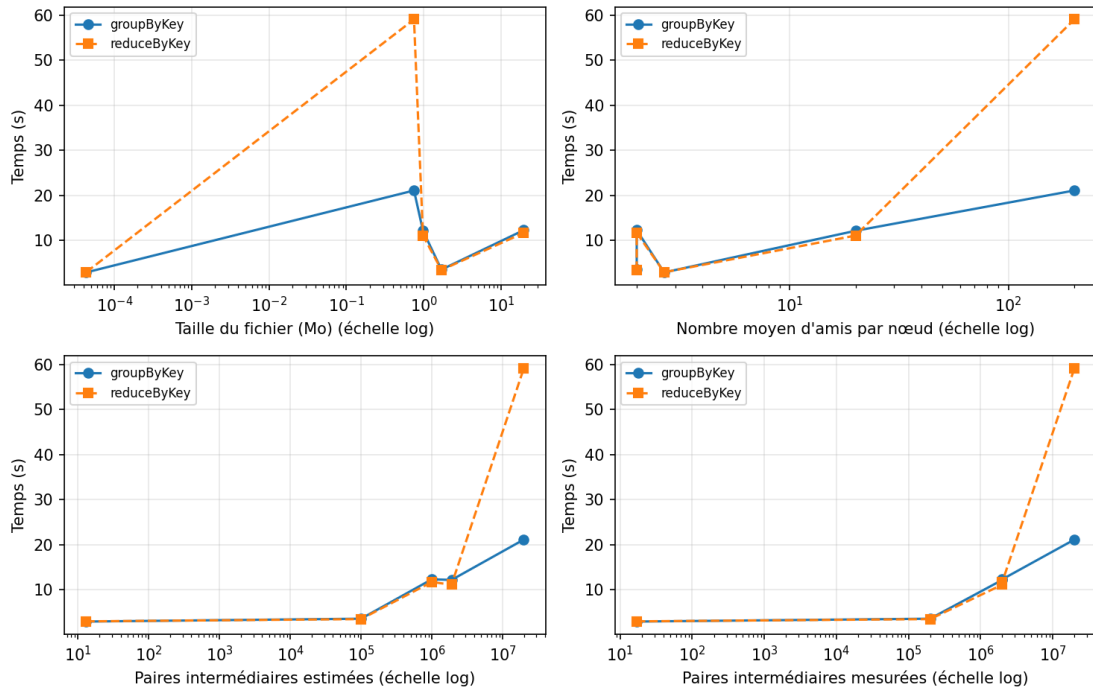


FIGURE 1 – Temps d'exécution en fonction des 4 variables (abscisses triées, échelle log).

En fonction de la taille du fichier, la courbe part dans tous les sens : **sn_1k_100k**, le plus petit fichier après **sn_tiny**, est le plus lent. Le nombre moyen d'amis ne suffit pas non plus, puisque **sn_100k_100k** et **sn_1m_1m** ont la même moyenne mais 3,5 s contre 12 s. Avec le nombre estimé de paires la courbe est presque croissante, avec une petite inversion entre **sn_1m_1m** et **sn_10k_100k**. C'est avec le nombre mesuré de paires intermédiaires que la progression est la plus régulière (strictement croissante pour les deux versions). C'est donc cette variable qui caractérise le mieux le temps d'exécution : le coût vient de la création, du transfert et du regroupement de ces paires, pas de la taille des données en entrée.

3 Moyenne et écart type

3.1 Implémentation

On utilise $\sigma = \sqrt{x^2 - \bar{x}^2}$, ce qui permet de tout obtenir à partir de trois sommes. Comme pour l'exercice 1, le script contient une version lente, qui construit la liste des températures de chaque année (et a donc les mêmes défauts qu'en 1.1), et une version rapide, que nous avons utilisée pour toutes les mesures :

```
def stddev_temperature_slow(theText_file):
    temperatures = theText_file.map(lambda line: line.split(",")) \
```

```

.map(lambda term: (term[0], [float(term[6])])) \
.reduceByKey(lambda x, y: x+y) \
.mapValues(lambda lv: (sum(lv)/len(lv),
                        sqrt(sum(t*t for t in lv)/len(lv) - (sum(lv)/len(lv))**2)))
return temperatures

def stddev_temperature_fast(theText_file):
    temperatures = theText_file.map(lambda line: line.split(",")) \
        .map(lambda term: (term[0], (float(term[6]), float(term[6])**2, 1))) \
        .reduceByKey(lambda x, y: (x[0]+y[0], x[1]+y[1], x[2]+y[2])) \
        .mapValues(lambda s: (s[0]/s[2], sqrt(s[1]/s[2] - (s[0]/s[2])**2)))
    return temperatures

```

Dans la version rapide, chaque mesure t devient $(t, t^2, 1)$ et le `reduceByKey` somme les trois composantes. On garde une valeur de taille fixe, donc la réduction se fait localement avant le shuffle comme en 1.2.

```

$ spark-submit --master spark://sar17:7077 stddev_temperatures.py temperatures_86400.csv 2>&1 | grep "Job 0 finished" | grep -o "took.*"
took 2935.296493 ms
$ hdfs dfs -cat hdfs://sar17:9000/sdim/sdim_9/temperatures_86400.out/* | sort | tail -6
('2013', (8.159945205479453, 12.095467213807215)) ('2016', (8.337049180327872, 11.508239269066483))
('2014', (7.41616438356164, 12.000489970606273)) ('2017', (7.345287671232872, 11.692122262422554))
('2015', (7.788328767123292, 11.422338877224695)) ('2018', (8.23512328767123, 11.458240836489342))

```

Les valeurs sont identiques à celles de l'énoncé.

3.2 Performances

```

$ for f in 2880 86 10; do hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/temperatures_${f}.out; echo -n "temperatures_${f} : "; spark-submit --master spark://sar17:7077 stddev_temperatures.py temperatures_${f}.csv 2>&1 | grep "Job 0 finished" | grep -o "took.*"; done
temperatures_2880 : took 3333.240596 ms temperatures_86 : took 5197.73217 ms temperatures_10 : took 24335.452009 ms

```

Fichier	86400	2880	86	10
Moyenne, version lente (s)	2,95	7,18	79,68	–
Moyenne, version rapide (s)	2,99	3,17	5,07	22,34
Moyenne + écart type (s)	2,94	3,33	5,20	24,34

TABLE 3 – Comparaison avec l'exercice 1.

Calculer l'écart type en plus ne coûte presque rien : au pire 9 % de plus que la moyenne seule (24,3 s contre 22,3 s sur 3 Go), et on reste très loin de la version lente. Le temps d'exécution dépend donc surtout de la façon dont le calcul est organisé (volume échangé lors du shuffle, taille des valeurs réduites, parallélisme de la réduction), et assez peu du nombre d'opérations faites sur chaque mesure. Ajouter un élément au tuple ne change ni la taille fixe des valeurs, ni le nombre de paires échangées.

4 Variation du nombre de cœurs

4.1 Mesures

On relance la version rapide de l'exercice 3 sur `temperatures_10.csv` (3 Go) en limitant le nombre total de cœurs, donc d'executors (un par nœud) :

```

$ for c in $(seq 3 15); do hdfs dfs -rm -r -f hdfs://sar17:9000/sdim/sdim_9/temperatures_10.out; echo -n "$c cores : "; spark-submit --total-executor-cores $c --master spark://sar17:7077 stddev_temperatures.py temperatures_10.csv 2>&1 | grep "Job 0 finished" | grep -o "took.*"; done
3 cores : took 87995.839042 ms 6 cores : took 45315.323015 ms 9 cores : took 34544.823586 ms 12 cores : took 24355.716773 ms
4 cores : took 66148.261573 ms 7 cores : took 44590.202644 ms 10 cores : took 34255.370815 ms 13 cores : took 24360.584403 ms
5 cores : took 55448.511994 ms 8 cores : took 34313.007765 ms 11 cores : took 33981.879857 ms 14 cores : took 24004.90604 ms
15 cores : took 23990.139736 ms

```

Nœuds	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Temps (s)	250,9	128,5	88,0	66,1	55,4	45,3	44,6	34,3	34,5	34,3	34,0	24,4	24,4	24,0	24,0
Accélération	1,0	2,0	2,9	3,8	4,5	5,5	5,6	7,3	7,3	7,3	7,4	10,3	10,3	10,5	10,5

TABLE 4 – Temps et accélération T_1/T_n (valeurs pour 1 et 2 nœuds fournies dans TP1-Perf.xlsx).

4.2 Analyse

La courbe nous paraît logique. Jusqu'à 5 ou 6 nœuds, l'accélération est quasiment idéale : chaque bloc est traité indépendamment, et le shuffle final est minuscule (un triplet par année et par partition). Ensuite, le temps baisse par paliers : il ne bouge pas entre 6 et 7 nœuds, entre 8 et 11, ni entre 12 et 15.

Ces paliers viennent du découpage du fichier. Avec des blocs HDFS de 128 Mo, le fichier de 3 Go donne 24 partitions, donc 24 tâches, ce que confirment les logs :

```

$ spark-submit --master spark://sar17:7077 stddev_temperatures.py temperatures_10.csv 2>&1 | grep -iE "missing tasks"
INFO Submitting 24 missing tasks from ShuffleMapStage 0 (PairwiseRDD[3] at reduceByKey ...)
INFO Submitting 24 missing tasks from ResultStage 1 (MapPartitionsRDD[8] at saveAsTextFile ...)

```

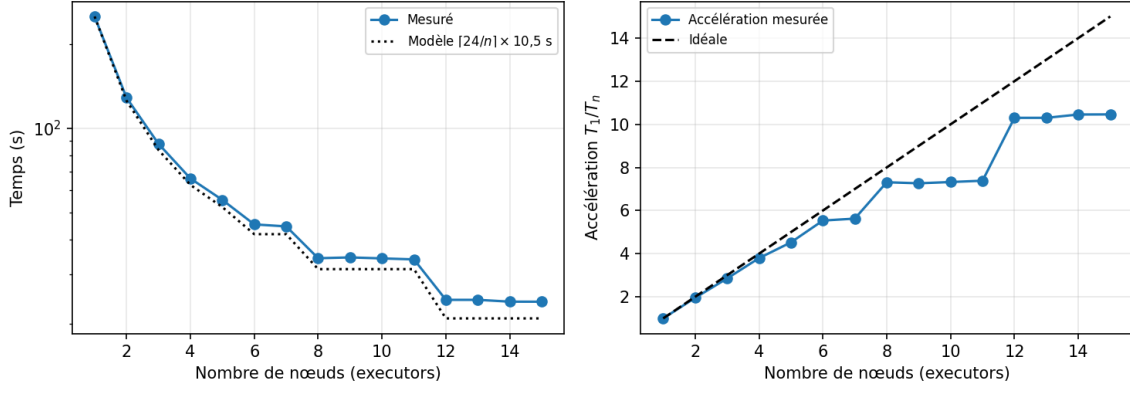


FIGURE 2 – Temps d'exécution (échelle log) et accélération en fonction du nombre de nœuds.

Avec n executors d'un cœur, les 24 tâches passent en $\lceil 24/n \rceil$ vagues d'environ $T_1/24 \approx 10,5$ s. Cela donne 8 vagues pour 3 nœuds, 6 pour 4, 5 pour 5, 4 pour 6 et 7, 3 de 8 à 11, et 2 de 12 à 15. Ce modèle simple (pointillés sur la figure) suit très bien les mesures ; les 1 à 3 s d'écart correspondent au coût fixe du job (lancement, shuffle, écriture). Un nœud supplémentaire n'aide donc que s'il fait baisser le nombre de vagues : au-delà de 12, des executors restent inactifs pendant la deuxième vague. Pour profiter des 15 nœuds, il faudrait un nombre de partitions multiple de 15, par exemple avec `sc.textFile(..., minPartitions=30)`.